

Requirements — Functional & Non-Functional

Project: Aura Promptly
Version: 1.1
Date: 2026-05-14
Status: Baselined (updated post-review)

Change Log

Version	Date	Changes
1.0	2026-05-14	Initial baseline
1.1	2026-05-14	Updated NFR-01 cost targets (pgvector replaces OpenSearch); added NFR-27/28 two-layer observability; added NFR-29–34 public repo security controls; updated FR-26 GCP auth; updated FR-35/36 observability layering; split FR-48/FR-53 CI/CD phasing

1. Background & Objectives

1.1 Purpose

Aura Promptly is a PoC to validate and demonstrate a **centralised AI governance model** for enterprises operating across multiple cloud providers. The central hub account (AWS) owns all AI governance controls while application teams in Azure, AWS and GCP consume AI capabilities through a single, governed API gateway.

1.2 Goals

ID	Goal
G-01	Demonstrate central governance of AI models, spend, guardrails and observability
G-02	Prove multi-cloud application connectivity without requiring private network links
G-03	Produce a reusable, Terraform-managed infrastructure that can be destroyed and recreated at will to minimise cost
G-04	Serve as educational material for a LinkedIn article and YouTube video series (public GitHub repository)
G-05	Establish the architectural pattern a real enterprise could adopt as a baseline

1.3 PoC Scope Boundary

- **In scope:** Hub AWS account, 3 spoke sample applications (Azure, AWS, GCP), LiteLLM gateway, Bedrock inference + RAG, Bedrock Agent, observability, guardrails, CI/CD pipeline, cost tooling
- **Out of scope:** Production workloads, real customer data, HIPAA/PCI data processing, private network peering (VPN/ExpressRoute), fine-tuning

2. Stakeholders

Role	Responsibility
Platform Engineer (PoC author)	Design, build, maintain, document the platform
Application Developer (simulated)	Consume the platform via sample apps in each cloud
Central AI Governance Team (simulated)	Set guardrails, review observability dashboards, manage spend budgets

3. Functional Requirements

3.1 Central Hub — AI Gateway

ID	Requirement	Priority
FR-01	The platform SHALL expose a single API endpoint that application teams call for all AI model inference	MUST
FR-02	The gateway SHALL route requests to different backend models (Bedrock, OpenAI, Vertex AI) based on configurable routing rules	MUST
FR-03	The gateway SHALL support load balancing across multiple model endpoints of the same provider	SHOULD
FR-04	The gateway SHALL enforce per-team and per-model spend budgets, blocking requests when limits are exceeded	MUST
FR-05	The gateway SHALL track token usage per application team and per model	MUST
FR-06	The gateway SHALL support hot-reload of routing configuration without downtime	SHOULD
FR-07	The gateway SHALL expose an admin UI for viewing spend, usage and routing configuration	SHOULD

3.2 Model Access — Amazon Bedrock

ID	Requirement	Priority
FR-08	The platform SHALL provide access to Amazon Bedrock Claude Haiku 3 as the primary PoC model	MUST

ID	Requirement	Priority
FR-09	The platform SHALL provide Amazon Titan Text Embeddings v2 for RAG embedding generation	MUST
FR-10	The Terraform configuration SHALL include commented-out blocks for other Bedrock models (Claude Sonnet, Llama 3, Mistral) to serve as enablement guides	SHOULD
FR-11	All Bedrock calls SHALL be on-demand (no provisioned throughput) for cost minimisation	MUST

3.3 RAG — Knowledge Base

ID	Requirement	Priority
FR-12	The platform SHALL include a Bedrock Knowledge Base connected to an S3 data source	MUST
FR-13	The Knowledge Base SHALL support ingestion of web-scraped documents (uploaded to S3 as plain text/PDF)	MUST
FR-14	The design SHALL document integration patterns for Confluence and SharePoint as future data sources, with placeholder Terraform stubs	SHOULD
FR-15	The platform SHALL support Knowledge Base sync (manual trigger via GitHub Actions and/or on-demand API call)	MUST

3.4 Bedrock Agent

ID	Requirement	Priority
FR-16	The platform SHALL deploy a Bedrock Agent demonstrating a multi-step agentic "Research Assistant" workflow: querying the Knowledge Base and synthesising a structured report	MUST
FR-17	The Agent SHALL use at least one Action Group backed by a Lambda function (<code>search_knowledge_base</code>) that retrieves chunks from the Bedrock Knowledge Base	MUST
FR-18	The Agent SHALL be accessible via the LiteLLM gateway using the native <code>bedrock/agent/{AGENT_ID}/{ALIAS_ID}</code> route (no custom Lambda adapter required)	MUST

3.5 External Model Integration

ID	Requirement	Priority
FR-19	The gateway SHALL support routing to OpenAI GPT-4o-mini via the LiteLLM OpenAI provider	MUST
FR-20	The gateway SHALL support routing to Google Vertex AI Gemini Flash via the LiteLLM Vertex provider	MUST

ID	Requirement	Priority
FR-21	External provider API keys SHALL be stored in AWS Secrets Manager and injected into LiteLLM at runtime via <code>os.environ/</code> references in the config YAML	MUST
FR-22	A model fallback chain SHALL be configured: Bedrock Haiku → OpenAI GPT-4o-mini → error	SHOULD

3.6 Multi-Cloud Connectivity

ID	Requirement	Priority
FR-23	Application teams SHALL connect to the central hub over HTTPS only (no VPN/private link required)	MUST
FR-24	The hub SHALL expose an AWS API Gateway endpoint as the public-facing entry point	MUST
FR-25	Authentication SHALL support API keys (for Azure spoke); keys SHALL be stored in Azure Key Vault and never committed to the repository	MUST
FR-26	Authentication SHALL support GCP Workload Identity OIDC federation: Cloud Run service account OIDC token → AWS STS AssumeRoleWithWebIdentity → temporary IAM credentials → IAM Sig V4 call to API Gateway	MUST
FR-27	Authentication SHALL support IAM roles (for AWS-native spoke applications via cross-account STS)	MUST
FR-28	The API Gateway SHALL be protected by AWS WAF with rate limiting and common OWASP rule set	MUST

3.7 Guardrails

ID	Requirement	Priority
FR-29	The platform SHALL apply Bedrock Guardrails to all Bedrock model calls	MUST
FR-30	Guardrails SHALL block harmful content categories (hate speech, violence, profanity)	MUST
FR-31	Guardrails SHALL enforce topic denial for pre-configured off-topic subjects	SHOULD
FR-32	LiteLLM content filter callbacks SHALL be configured as a secondary guardrail layer for non-Bedrock models	SHOULD
FR-33	Guardrail violation events SHALL be logged and visible in the observability dashboard	MUST

3.8 Observability

ID	Requirement	Priority
FR-34	All inference requests and responses SHALL be logged (request ID, model, team, tokens, latency, guardrail result)	MUST
FR-35	Logs SHALL be shipped to CloudWatch Logs as the primary (always-on, no data egress) observability layer	MUST
FR-36	CloudWatch Dashboards SHALL provide the same 4 views as the Grafana dashboards: total requests, requests per team, requests per model, average latency, error rate, spend per team	MUST
FR-37	An optional secondary observability layer SHALL forward logs/metrics/traces via OTel Collector to Grafana Cloud, controlled by <code>enable_grafana_export</code> Terraform variable	SHOULD
FR-38	Traces SHALL be collected via OpenTelemetry and visualised in Grafana Tempo when <code>enable_grafana_export = true</code>	SHOULD
FR-39	A sample response log SHALL demonstrate the structure of logged model responses (no retention required for PoC)	MUST

3.9 Cost Tooling

ID	Requirement	Priority
FR-40	Terraform workspace SHALL integrate Infracost to produce a cost estimate before every <code>terraform apply</code> (from Phase 1b onwards)	MUST
FR-41	LiteLLM SHALL track per-team, per-model spend and expose a spend dashboard	MUST
FR-42	A <code>destroy</code> runbook SHALL be documented and automated via GitHub Actions to tear down all infrastructure in one command	MUST
FR-43	A <code>recreate</code> runbook SHALL restore full PoC infrastructure from scratch in one command, including a poll-and-wait step for Knowledge Base sync completion before deploying spokes	MUST

3.10 Sample Applications

ID	Requirement	Priority
FR-44	An Azure sample application SHALL demonstrate a RAG chatbot calling the hub endpoint (API key auth)	MUST
FR-45	An AWS spoke sample application SHALL demonstrate the Bedrock Agent "Research Assistant" workflow via the hub (IAM Sig V4 auth)	MUST
FR-46	A GCP sample application SHALL demonstrate a summarisation task using the hub endpoint (GCP Workload Identity → IAM Sig V4 auth)	MUST

ID	Requirement	Priority
FR-47	Each sample app SHALL include a minimal UI (Streamlit or Gradio) for demo purposes	SHOULD
FR-48	Each sample app SHALL be containerised and deployable via Terraform	MUST

3.11 CI/CD

ID	Requirement	Priority
FR-49	GitHub Actions SHALL be used for all CI/CD pipelines	MUST
FR-50	A basic pipeline (Phase 1b) SHALL run <code>terraform fmt -check</code> , <code>terraform validate</code> , and <code>terraform plan</code> on every pull request from Phase 1 onwards	MUST
FR-51	A basic pipeline (Phase 1b) SHALL run <code>terraform apply</code> on merge to <code>main</code> for the hub workspace	MUST
FR-52	The full pipeline (Phase 11) SHALL add Infracost diff, tfsec/checkov security scan, all-workspace plan/apply, Azure and GCP OIDC, destroy workflow, and KB sync workflow	MUST
FR-53	A pipeline SHALL support manual <code>terraform destroy</code> trigger via workflow dispatch	MUST
FR-54	GitHub Actions OIDC integration with AWS SHALL be used from Phase 1b (no long-lived AWS credentials in GitHub secrets)	MUST

3.12 Public Repository Security

ID	Requirement	Priority
FR-55	All Terraform outputs that reference secrets SHALL be marked <code>sensitive = true</code> ; secret values SHALL never appear in plan output, apply output, or Terraform Cloud UI	MUST
FR-56	The repository SHALL include a <code>.gitignore</code> covering <code>*.tfvars</code> , <code>terraform.tfstate</code> , <code>*.tfstate.backup</code> , <code>.terraform/</code> , and <code>override.tf</code> files	MUST
FR-57	GitHub secret scanning and push protection SHALL be enabled on the repository to block accidental credential commits	MUST
FR-58	A secret scanning step (gitleaks or trufflehog) SHALL run on every pull request as part of the CI pipeline	MUST
FR-59	LiteLLM config YAML SHALL reference secrets via <code>os.environ/SECRET_NAME</code> only; no literal API keys or passwords in the YAML file	MUST
FR-60	A <code>CODEOWNERS</code> file SHALL require review of all changes under <code>infra/</code>	SHOULD

ID	Requirement	Priority
FR-61	A SECURITY.md file SHALL document the responsible disclosure process and note that this is a PoC with no real credentials	MUST

4. Non-Functional Requirements

4.1 Cost

ID	Requirement	Target
NFR-01	PoC monthly AWS spend SHALL be minimised; all services SHALL be on-demand or serverless	< \$30/month when idle (hub running, KB destroyed), < \$100/month during active development
NFR-02	The infrastructure SHALL be fully destroyable and recreatable to avoid idle costs	Recreate time < 30 minutes (including KB sync poll-and-wait)
NFR-03	All Bedrock calls SHALL use the cheapest model adequate for the use case	Claude Haiku 3 as primary

Note: NFR-01 targets revised downward from v1.0 due to switch from OpenSearch Serverless (~\$172/month) to Aurora pgvector (~\$10–15/month) as the RAG vector store.

4.2 Security

ID	Requirement	Target
NFR-04	All data in transit SHALL be encrypted with TLS 1.2 minimum	100%
NFR-05	All data at rest SHALL be encrypted using AWS-managed KMS keys	100%
NFR-06	No AWS credentials SHALL be stored in GitHub repository	Enforced by OIDC
NFR-07	API keys for external providers SHALL be stored in AWS Secrets Manager, never in code or environment variables	100%
NFR-08	The API Gateway SHALL enforce TLS and reject HTTP requests	100%
NFR-09	IAM roles SHALL follow least-privilege principle; no wildcard resource ARNs in production policies	100%
NFR-10	AWS WAF SHALL protect the public API Gateway endpoint with rate limiting (100 req/5 min per IP for PoC)	Configurable
NFR-11	The platform design SHALL reference financial services baseline controls (aligned to NIST CSF) as a framework, without claiming full compliance	Reference only

4.3 Reliability

ID	Requirement	Target
NFR-12	LiteLLM on ECS Fargate SHALL have at least 1 task running at all times during PoC active sessions	1 task minimum
NFR-13	Bedrock model fallback SHALL be configured to handle model unavailability	Haiku → GPT-4o-mini
NFR-14	The Knowledge Base sync process SHALL be idempotent (Bedrock KB re-embeds only changed documents on re-sync)	100%
NFR-15	The recreate runbook SHALL poll the Bedrock ingestion job status and wait for COMPLETE before deploying spokes	100%

4.4 Operability

ID	Requirement	Target
NFR-16	Every deployment step SHALL be documented with novice-level instructions	100% coverage
NFR-17	All Terraform modules SHALL include a README.md with inputs, outputs and usage example	100% of modules
NFR-18	Terraform state SHALL be stored remotely (Terraform Cloud free tier)	100%
NFR-19	Terraform workspaces SHALL be separated by component (hub, azure-spoke, aws-spoke, gcp-spoke)	Yes

4.5 Maintainability

ID	Requirement	Target
NFR-20	Infrastructure code SHALL be modular — each concern in its own Terraform module	Yes
NFR-21	Application code SHALL be Python 3.11+	Yes
NFR-22	Dependency versions SHALL be pinned in all Terraform and Python files	Yes
NFR-23	The LiteLLM Docker image SHALL be pinned to a specific –stable semver tag (e.g. v1.83.0–stable); main–latest SHALL NOT be used	Yes

4.6 Scalability (design intent, not PoC target)

ID	Requirement	Note
----	-------------	------

ID	Requirement	Note
NFR-24	LiteLLM ECS service SHALL be configured for auto-scaling (disabled in PoC, documented for production)	Design intent
NFR-25	The Knowledge Base design SHALL support multiple data source connectors as a future capability	Stubs only

4.7 Observability

ID	Requirement	Target
NFR-26	Latency per inference call SHALL be visible in dashboards	P50, P95
NFR-27	Error rates SHALL trigger CloudWatch alarms (email notification only for PoC)	Yes
NFR-28	Cost per team SHALL be visible in LiteLLM dashboard and CloudWatch/Grafana	Yes
NFR-29	CloudWatch SHALL be the primary observability store; all logs and metrics SHALL be available in CloudWatch regardless of whether Grafana export is enabled	100%
NFR-30	The Grafana export layer SHALL be documented as optional for regulated environments where data egress is not permitted; the article/video SHALL explicitly note this	Yes

5. Assumptions

ID	Assumption
A-01	All three spoke cloud accounts start as empty/vanilla (no existing VPCs, IAM, or services)
A-02	The PoC developer has admin access to all three cloud provider accounts
A-03	Terraform Cloud free tier (1 user, 500 managed resources) is sufficient for this PoC
A-04	Grafana Cloud free tier (10k metrics, 50GB logs) is sufficient for the optional Grafana export layer
A-05	OpenAI API and Google Cloud accounts will be created fresh as part of the PoC
A-06	GitHub repository is public — all security controls in FR-55–FR-61 are mandatory
A-07	No real customer or employee data will be processed through the PoC

ID	Assumption
A-08	Bedrock model access in the chosen region must be requested and approved before deployment (can take up to 24 hours — request early)
A-09	Aurora Serverless pgvector is GA as a Bedrock Knowledge Base vector store (confirmed May 2026)
A-10	LiteLLM natively supports Bedrock Agent invocation via <code>bedrock/agent/{AGENT_ID}/{ALIAS_ID}</code> route (confirmed May 2026)

6. Constraints

ID	Constraint
C-01	AWS is the mandated central hub cloud provider
C-02	No private network links (VPN, ExpressRoute, Interconnect) — all spoke connectivity is over internet
C-03	GitHub Actions only for CI/CD (no Jenkins)
C-04	Infrastructure must be expressible 100% in Terraform
C-05	PoC is maintained by a single developer on a local machine
C-06	No provisioned throughput / reserved capacity for Bedrock
C-07	GitHub repository is public — no secrets, credentials, or sensitive outputs may appear in any committed file or Terraform output

7. Out of Scope

- PII detection or redaction
- HIPAA, PCI-DSS or SOC2 certification
- Private network connectivity (VPN, ExpressRoute, Interconnect)
- Fine-tuning of foundation models
- Production SLAs or uptime commitments
- Multi-region active-active deployment
- Real Confluence or SharePoint integration (stubs only)